CSE113: Parallel Programming

Homework 2: Mutexes
Assigned: October 8, 2026
Due: October 18, 2026

Preliminaries

1. This assignment requires the following programs: `make`, `bash`, `python3`, and `g++`. Please use the Docker image provided for this course to ensure that your code runs in the same environment used by the teaching team for grading. More instructions on how to use the docker image are provided at the end of this document.
2. If you need help, please visit office or mentoring hours. You can also ask questions on Piazza. You are allowed to ask your classmates questions about frameworks and languages (e.g. Docker, Git, and C++). You are not allowed to discuss homework solution details with them.
3. In this assignment, you will be provided with a template code that you will need to complete. You will be graded on the correctness of your code, the performance of your code, and the quality of your report.
4. **Your submission will consist of two items**: A PDF report, and the completed codes. Zip your code and PDF report into a single file and submit it to Canvas.

0.1 Structure of the report:

Create one PDF containing 3 sections, one for each part of the assignment. For each part of the assignment, your report should contain the following sub-sections:

- **Design**: describe the design of your code/solution in enough detail that a knowledgeable programmer could duplicate your work. This includes descriptions of the workflow of each part and a description of each function with its purpose, inputs, outputs, and assumptions it makes about inputs or outputs.
- **Results**: Read each part of the assignment for more details on what to include in your results.
- **Analysis**: Analyze your results. Why do you think you got the results you did? What trend do you observe? Any unexpected results? Does the result agree with what you expected? If not, what could be the reason?

If you want help with your code, you have to share your **Design** part of the report with us. We can't debug your code without knowing what you are trying to do.

There is a template for the report provided in the assignment named **report.tex**. You can use the template to structure your report but you are not required to use it.

1 Implementing Mutexes

In this part of the assignment you will implement two mutex variants. The first is the filter lock, an N threaded generalization of Peterson's algorithm. You can find the algorithm description in section 2.4 of the book. The second mutex you will implement is Lamport's Bakery Algorithm. It is given in section 2.6 of the book.

You will measure the throughput and fairness of each mutex using the skeleton C++ code provided in the packet. The main function repeatedly locks and unlocks a mutex on each thread for 1 second. It reports the throughput of the mutex overall (how many total mutex lock/unlocks occurred) and a histogram for how many times each thread obtained the mutex.

Your work is constrained to the two mutex header files: `filter.h` and `bakery.h`. You must implement 4 functions:

- the constructor
- `init`: an initialization function that is called before threads are launched. It takes in the number of threads as an argument.
- `lock`: as described in lecture. It takes the thread id as an argument.
- `unlock`: as described in lecture. It also takes a thread id as an argument.

You must also provide the necessary private variables to implement the mutex. You should use atomic data types (only when required), and **you must use the `store` and `load` methods to access memory through the atomics.** *You are not allowed to use atomic RMWs in any part of your implementations!*

You are allowed to use the book as a reference, as well as the class lectures/slides. Please do not explicitly search online for C++ implementations of these mutexes.

You can gain confidence in your implementations by running them with the thread sanitizer. That is, add the following command line option `-fsanitize=thread`, and then execute the program. This is done for you if you run `make test`. Your mutex implementations should execute without errors.

The make file produces three executables, one for each lock:

- `cpp_mutex`: which uses the C++ mutex object.
- `filter_mutex`: which uses the mutex from `filter.h`
- `bakery_mutex`: which uses the mutex from `bakery.h`

1.1 Experiments

Once your locks are implemented, you will run the three executables with various configurations and record the results. There are two results you will be gathering:

- **the throughput**: this is recorded as the `total number` value that is printed out. It records how many locks/unlocks occurred across all threads.

- **the fairness**: we need some way how to measure how fair the implementation is. For this, we will use a statistical metric called *coefficient of variation*[1]. While the math can be a bit tricky, the high-level idea can be expressed as:

  > It shows the extent of variability in relation to the mean of the population.

  There is an online calculator you can use to compute this statistic here: `https://www.socscistatistics.com/descriptive/coefficientvariation/default.aspx`

  This metric gives a number you can use to compare fairness across your different implementations, even if they have different throughputs. The lower this number is, the more fair the mutex is.

You should run and report on the following experiments:

- Run with as many threads as you have cores. Record each mutex's throughput and the thread fairness using the coefficient of variance (for each mutex). For the throughput average, you can exclude outliers.
- Run with 2 times as many cores as your machine. Record each mutex's throughput and the thread fairness using the coefficient of variance (for each mutex). For the throughput average, you can exclude outliers.
- Run with 4 times as many cores as your machine. Record each mutex's throughput and the thread fairness using the coefficient of variance (for each mutex). For the throughput average, you can exclude outliers.

These results should be gathered without thread sanitizer enabled.

1.2 Adding Yield

For the next part of the assignment, add a `yield` to the spin loop of both of your mutex implementations (the filter and bakery mutexes).

Repeat the experiments with your new mutex implementations using yield.

1.3 What to Submit

The code component of your submission is the completed header files for the filter lock and bakery lock (`filter.h` and `bakery.h`). Please submit the versions with the yield included.

The report component consists of your mutex results, both the throughput and coefficient of variance. Please list your results in a table. Please provide the historgram data for your some of your experiments and comment on any outliers. Do your observations change if you manually remove some of the outliers? What could cause the outliers? Write one or two paragraphs explaining your results, and how they compare to the C++ mutex.

Your grade will be based on 4 criteria:

- Correctness: Do your mutexes provide mutual exclusion?

[1] see the wikipedia page here: `https://en.wikipedia.org/wiki/Coefficient_of_variation`

- Conceptual: Do your implementations use atomic operations correctly (and as specified in this document) and did you implement the algorithms faithfully? Please comment your code.
- Performance: Do your throughput/fairness results match roughly what they should?
- Explanation: Do you explain your results accurately based on our lectures?

2 A Fair Reader-Writer Lock

The shortcoming of basic Reader-Writer mutex implementations is that they can potentially starve writers if enough readers continually access the mutex.

As you might have suspected, I have written a benchmark that does exactly that. It is a similar wrapper to Part 1, with the exception that we have 6 readers and 2 writers. The wrapper records the throughput of the readers and of the writers. You will notice that the writers obtain the mutex much less frequently than the readers.

Your job is to develop a scheme that provides more fairness to the writer. Your results should show a significant increase in the number of times that the writer is able to obtain the mutex. You should also notice that the readers will suffer in throughput.

Your implementation is constrained to `fair_mutex.h`. Your solution must not allow data conflicts in the critical section of the RWMutex, i.e. the writers must have exclusive access when they acquire the mutex. Additionally, your implementation may not contain any data conflicts. You can check this with the thread sanitizer, similar to Part 1.

As two additional constraints: your solution cannot reduce the throughput of readers when there are no writers. Your solution cannot use C++ condition variable objects.

2.1 Experiments

Record the throughput of the readers and writers as provided by the original code. Then record the throughput of the readers and writers as reported by your modifications. Run your code with 6 readers and 2 writers, and also with 8 readers and no writers. Your fair mutex should not significantly reduce the throughput of readers from the default mutex (i.e., when there aren't any writers). Please run the code both on your own machine and in the course Docker container.

There are two possible challenges for you to investigate if you are interested:

- try to increase the throughput of both the default mutex and the fair mutex
- try to make your fair mutex tunable, e.g. so that it is more or less fair to the writers depending on some parameter

2.2 What to Submit

The code component is the completed `fair_mutex.h`.

The report component consists of your results in a table. Please graph the readers and writers when executed with the default (6 2) configuration. Write one paragraph describing your solution in design section, and another paragraph analyzing your results in analysis section.

Your grade will be based on 4 criteria:

- Correctness: Does your mutex provide mutual exclusion? That is, there should be no writer-writer overlaps in critical sections, nor writer-reader conflicts. Your mutex *should* allow multiple readers in the critical section.
- Conceptual: does your implementations use a conceptually sound strategy to increase fairness to writers?
- Fairness: does your solution actually provide more fairness for the writers?
- Explanation: do you explain your results accurately based on our lectures.

3 A Concurrent Linked List

In the homework packet, I have provided a sequential stack implementation using a linked list: your homework is to use C++ mutexes to make this structure safe to access concurrently.

You will be modifying this code, however, the structure of the stack must remain the same: that is, each operation starts at the beginning node and traverses the list until the end. As a further constraint, the only values that will be pushed are integers between 0 and 2, inclusive. Peek and pop return -1 when the stack is empty.

There is a wrapper benchmark file that calls the stack methods concurrently and records the throughput. You have three implementations you need to provide. Similar to part 1, the makefile will produce executables for each one.

3.1 Coarse-grained locking

In this implementation (`coarse_lock_stack.h`), you should add a single mutex to the class's private variables. You should perform locking and unlocking for each of the three public methods.

3.2 RW locking

In this implementation (`rw_lock_stack.h`), you should use the C++ shared_mutex object. You should identify when you need to use the full lock, and when you can use the reader lock. Recall that the reader lock call is `lock_shared` and `unlock_shared`.

3.3 SwapTop

Building on your (`rw_lock_stack.h`) implementation, for the final file (`swaptop_stack.h`), you need to implement a new API function called `swaptop` (swap top). This function takes in an integer and swaps the top element of the stack with the new value. Performing SwapTop on an empty stack simply returns without modifying the stack.

You should think about how to optimize this function using a read lock. That is, there are parts of the function that can be efficiently implemented just with the read lock. As a hint, please recall that the stack contains only 3 possible values: 0,1,2.

The rest of this third implementation should be the same as the RW locking.

All of your solutions should be free from data-conflicts. That is, they should pass Clang's thread sanitizer check when executing the wrapper. Your modifications to the stack should not effect its single threaded behavior (i.e. a push should add an element to the stack: a subsequent pop should retrieve the element).

3.4 Experiments

Run each of your 3 executables and record the throughputs of the different operations. Due to timing variations, please run each experiment 3 times and report the average, and comment on the spread of values you see during runs.

3.5 What to Submit

The code component of your submission is the completed header files.

In the report component please give the results of your experiments using a table. Write 1 paragraph about your solution to SwapTop, and how you used the RW lock in the design section. Write 2 paragraphs analyzing your results in the analysis section.

Your grade will be based on 4 criteria:

- Correctness: Is your list free from data conflicts and does it maintain the correct behavior of a stack?
- Conceptual: Are the methods correctly locked with the different mutexes? Does the SwapTop function efficiently provide its functionality?
- Performance: do your performance results match roughly what they should? Do your RW locks provide higher throughput?
- Explanation: do you explain your results accurately based on our lectures?

4 Docker Helper

Your homework must run in the course Docker image so that it uses the same environment as used by the teaching team. The image provides a simple Ubuntu environment with `python3`, C++ compilers, `make`, `vim`, and `emacs`. Even if the assignment runs on your Linux, macOS, or WSL installation, test it in the course image before submitting; code that does not run in the image cannot be graded.

4.1 Install Docker

Install and start Docker before continuing. Linux users should follow the official Docker Engine installation guide for Linux: https://docs.docker.com/engine/install/. macOS users should follow https://docs.docker.com/desktop/setup/install/mac-install/, and Windows users should follow https://docs.docker.com/desktop/setup/install/windows-install/.

Verify that Docker is running with:

```
docker --version
docker run --rm hello-world
```

4.2 Run the Course Container

First download the course image:

```
docker pull reeselevine/cse113:latest
```

From the top-level directory of your assignment, start a container and mount that directory at `/assignments`. The mounted directory keeps your assignment files on your computer, so edits made inside or outside the container are visible in both places.

On Linux, macOS, or WSL, run:

```
docker run --name cse113 -v "$(pwd)":/assignments -it reeselevine/cse113:latest bash
```

In Windows PowerShell, run:

```
docker run --name cse113 -v ${pwd}:/assignments -it reeselevine/cse113:latest bash
```

In Windows Command Prompt, run:

```
docker run --name cse113 -v %cd%:/assignments -it reeselevine/cse113:latest bash
```

Inside the container, use `cd /assignments` to access the assignment files. Run the commands specified in each part of the homework, and type `exit` when you are finished.

After the first run, reopen the same container with:

```
docker start -ai cse113
```

To see existing containers, use `docker ps -a`. Do not store important work only inside a container—only files in the mounted `/assignments` directory are kept with your homework on the host machine.